

FleetSpark — System Architecture Document

Version: 1.0

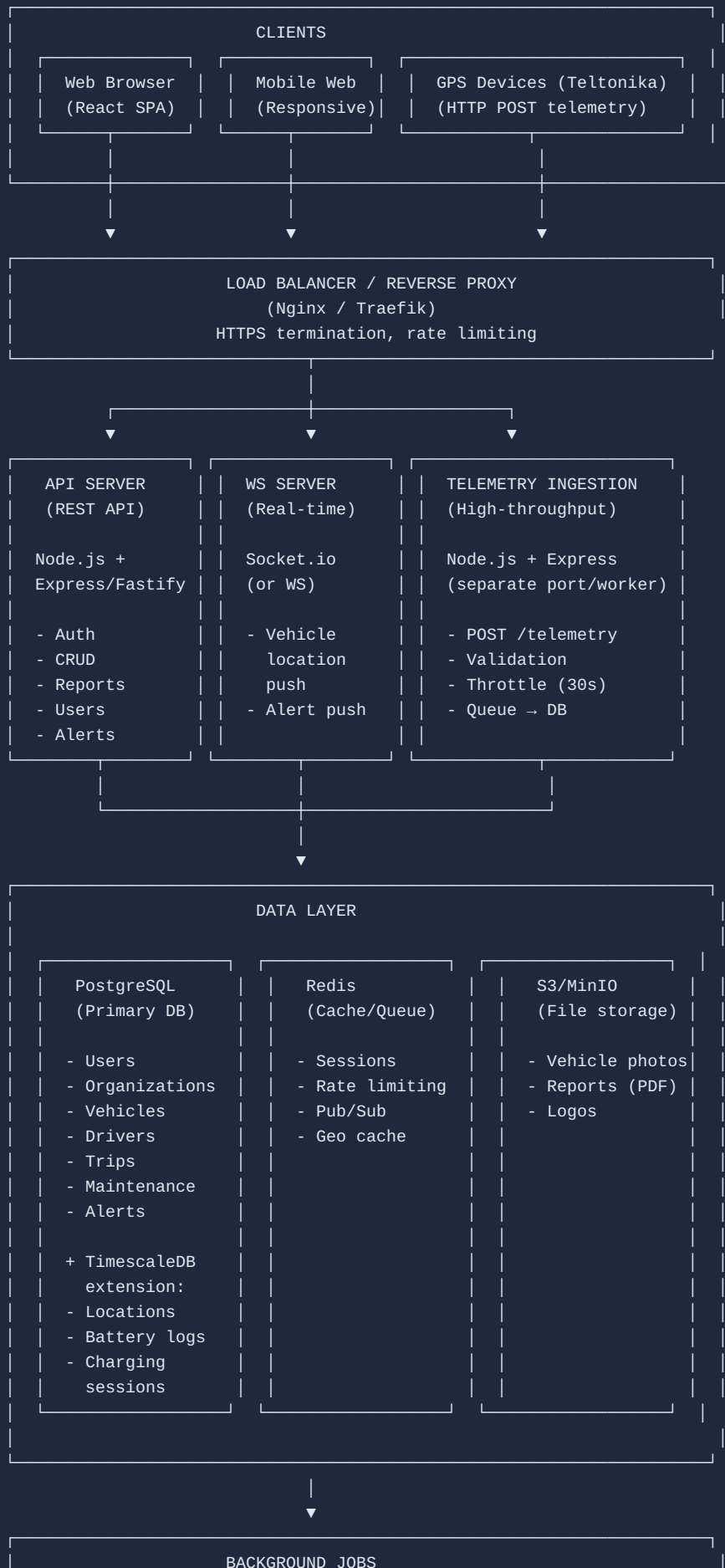
Date: June 11, 2026

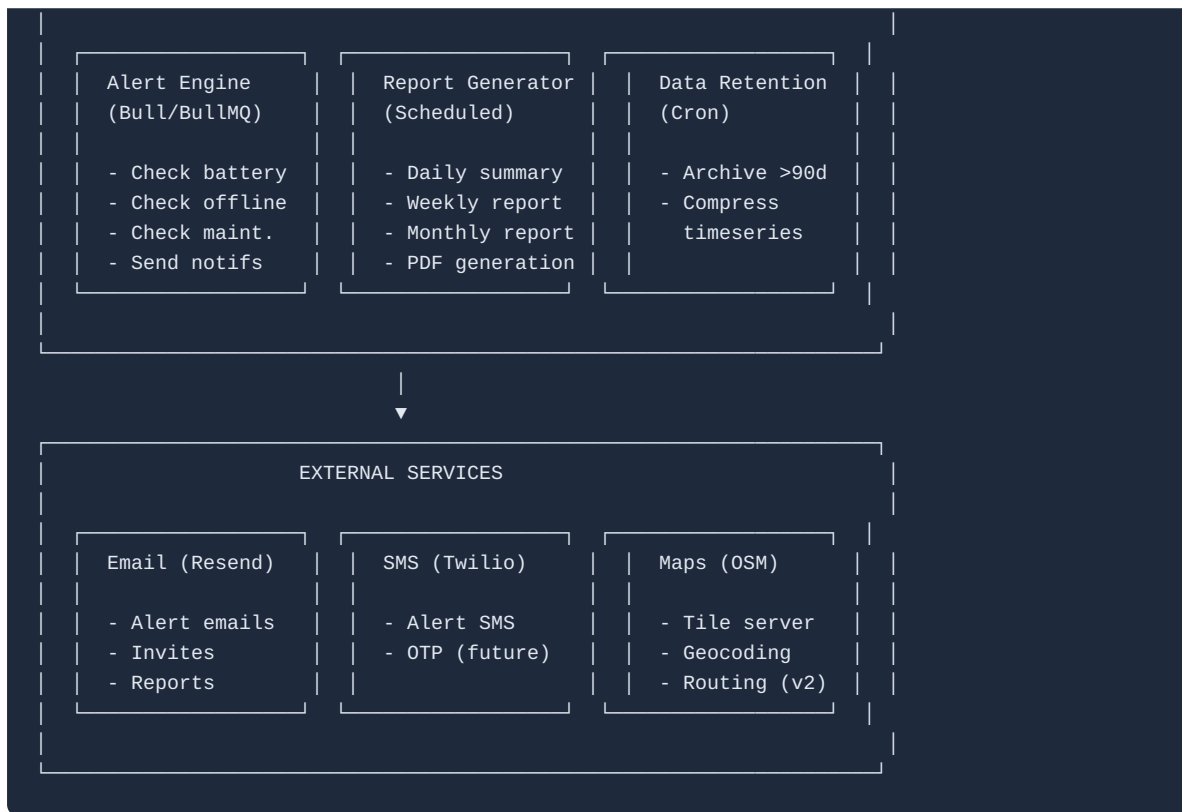
Author: SyncSphere CEO

Status: Draft — For Dev implementation

1. Architecture Overview

High-Level Diagram





2. Technology Stack

Backend

Component	Technology	Rationale
Runtime	Node.js 22 LTS	Async I/O, large ecosystem, JSON-native
Framework	Express.js (or Fastify)	Mature, middleware ecosystem, easy to hire for
ORM	Prisma	Type-safe DB queries, migrations, excellent DX
Validation	Zod	Runtime validation, TypeScript integration
Auth	bcrypt + JWT (jsonwebtoken)	Industry standard, stateless
WebSocket	Socket.io	Fallback support (polishing), rooms for org isolation
Queue	BullMQ (Redis-backed)	Reliable job processing, retries, scheduling
Logging	pino	Fast, structured JSON logs
Testing	Vitest + Supertest	Fast, modern, Jest-compatible API

Frontend

Component	Technology	Rationale
Framework	React 18+	Component model, huge ecosystem
Language	TypeScript	Type safety, fewer runtime errors
Build	Vite	Fast HMR, optimized builds
UI Library	Tailwind CSS + Headless UI	Rapid development, accessible
State Management	Zustand	Lightweight, no boilerplate
Data Fetching	TanStack Query (React Query)	Caching, refetching, optimistic updates
Maps	Leaflet + React-Leaflet	Free, works offline, OSM tiles
Charts	Chart.js + react-chartjs-2	Simple, responsive, good defaults
Forms	React Hook Form + Zod	Performant, type-safe validation
Routing	React Router v6	Standard, nested routes
Notifications	Hot Toast	Lightweight, customizable

Database

Component	Technology	Rationale
Primary	PostgreSQL 16	Reliable, JSON support, full-text search
Time-series	TimescaleDB extension	Optimized for telemetry (locations, battery logs)
Cache	Redis 7	Sessions, rate limiting, pub/sub, job queue
File Storage	MinIO (self-hosted S3)	S3-compatible, no egress costs

Infrastructure

Component	Technology	Rationale
Hosting	Coolify on EC2	SyncSphere's existing stack, Docker-native
Reverse Proxy	Traefik (via Coolify)	Auto HTTPS, load balancing
CI/CD	GitHub Actions → Coolify	Automated deploy on push
Monitoring	Uptime Kuma + pino logs	Uptime tracking, structured logging
Backups	pg_dump cron + S3	Daily DB backups, 30-day retention

3. Project Structure

```

fleetspark/
├── .github/
│   └── workflows/
│       └── deploy.yml                # CI/CD pipeline
├── prisma/
│   ├── schema.prisma                # Database schema
│   ├── migrations/                  # Prisma migrations
│   └── seed.ts                       # Seed data (demo org, vehicles)
├── src/
│   ├── server/
│   │   ├── index.ts                 # Entry point
│   │   ├── app.ts                   # Express app setup
│   │   ├── config.ts                # Environment config
│   │   ├── middleware/
│   │   │   ├── auth.ts              # JWT verification
│   │   │   ├── rateLimiter.ts       # Rate limiting
│   │   │   ├── validator.ts         # Request validation
│   │   │   └── errorHandler.ts      # Global error handler
│   │   ├── routes/
│   │   │   ├── auth.routes.ts       # POST /login, /register, /refresh
│   │   │   ├── vehicles.routes.ts   # CRUD /api/v1/vehicles
│   │   │   ├── drivers.routes.ts    # CRUD /api/v1/drivers
│   │   │   ├── trips.routes.ts      # GET /api/v1/trips
│   │   │   ├── battery.routes.ts    # GET /api/v1/battery/:vehicleId
│   │   │   ├── alerts.routes.ts     # CRUD /api/v1/alerts
│   │   │   ├── maintenance.routes.ts # CRUD /api/v1/maintenance
│   │   │   ├── reports.routes.ts    # GET /api/v1/reports
│   │   │   ├── users.routes.ts      # CRUD /api/v1/users
│   │   │   ├── telemetry.routes.ts   # POST /api/v1/telemetry
│   │   │   └── settings.routes.ts    # GET/PUT /api/v1/settings
│   │   ├── services/
│   │   │   ├── auth.service.ts      # Auth logic
│   │   │   ├── vehicle.service.ts   # Vehicle CRUD + queries
│   │   │   ├── telemetry.service.ts  # Telemetry ingestion + processing
│   │   │   ├── alert.service.ts      # Alert evaluation + dispatch
│   │   │   ├── report.service.ts     # Report generation
│   │   │   ├── notification.service.ts # Email + SMS sending
│   │   │   └── websocket.service.ts  # Socket.io management
│   │   ├── jobs/
│   │   │   ├── alert-check.job.ts    # Runs every 5 min
│   │   │   ├── offline-check.job.ts  # Runs every 10 min
│   │   │   ├── report-daily.job.ts   # Runs daily at 6 AM EAT
│   │   │   └── data-retention.job.ts  # Runs weekly
│   │   ├── lib/
│   │   │   ├── prisma.ts             # Prisma client singleton
│   │   │   ├── redis.ts              # Redis client
│   │   │   ├── email.ts              # Resend client
│   │   │   ├── sms.ts                # Twilio client
│   │   │   └── logger.ts              # pino logger
│   │   └── types/
│   │       ├── express.d.ts          # Express type extensions
│   │       └── telemetry.ts           # Telemetry payload types
│   └── client/                       # React frontend
│       ├── index.html
│       ├── src/
│       │   ├── main.tsx              # React entry
│       │   ├── App.tsx               # Root component + routing
│       │   ├── api/
│       │   │   ├── client.ts         # Axios instance + interceptors
│       │   │   ├── auth.api.ts       # Auth API calls
│       │   │   ├── vehicles.api.ts   # Vehicle API calls
│       │   │   ├── telemetry.api.ts  # Telemetry API calls
│       │   │   └── reports.api.ts     # Reports API calls

```

```

├── hooks/
│   ├── useAuth.ts          # Auth state + login/logout
│   ├── useVehicles.ts      # Vehicle list + CRUD
│   ├── useTelemetry.ts     # Real-time telemetry via WS
│   ├── useAlerts.ts        # Alerts list + acknowledge
│   └── useReports.ts        # Reports data + export
├── stores/
│   ├── auth.store.ts       # Auth Zustand store
│   ├── vehicle.store.ts    # Vehicle Zustand store
│   └── alert.store.ts      # Alert Zustand store
├── components/
│   ├── layout/
│   │   ├── Sidebar.tsx     # Navigation sidebar
│   │   ├── Header.tsx      # Top bar with user menu
│   │   └── PageWrapper.tsx # Page layout wrapper
│   ├── dashboard/
│   │   ├── SummaryCards.tsx # Fleet summary cards
│   │   ├── FleetMap.tsx     # Leaflet map with markers
│   │   └── VehicleList.tsx  # Vehicle table
│   ├── vehicles/
│   │   ├── VehicleForm.tsx  # Add/edit vehicle form
│   │   ├── VehicleDetail.tsx # Vehicle detail page
│   │   ├── BatteryGauge.tsx # Battery level gauge
│   │   └── TripHistory.tsx  # Trip list + map
│   ├── alerts/
│   │   ├── AlertList.tsx    # Alert table
│   │   └── AlertSettings.tsx # Alert threshold config
│   ├── reports/
│   │   ├── ReportChart.tsx  # Chart.js wrapper
│   │   └── ReportExport.tsx # PDF/CSV export buttons
│   └── ui/
│       ├── Button.tsx       # Reusable button
│       ├── Modal.tsx        # Reusable modal
│       ├── Table.tsx        # Reusable table
│       ├── Badge.tsx        # Status badge
│       └── Spinner.tsx      # Loading spinner
├── pages/
│   ├── Login.tsx
│   ├── Dashboard.tsx
│   ├── Vehicles.tsx
│   ├── VehicleDetail.tsx
│   ├── Drivers.tsx
│   ├── Alerts.tsx
│   ├── Reports.tsx
│   └── Settings.tsx
├── utils/
│   ├── format.ts           # Date, currency, number formatting
│   ├── constants.ts        # App constants
│   └── helpers.ts          # Utility functions
├── styles/
│   └── index.css           # Tailwind imports + custom CSS
├── vite.config.ts
├── scripts/
│   ├── seed-demo-data.ts   # Generate demo vehicles + telemetry
│   └── simulate-telemetry.ts # GPS simulator for testing
├── tests/
│   ├── server/
│   │   ├── auth.test.ts
│   │   ├── vehicles.test.ts
│   │   └── telemetry.test.ts
│   └── client/
│       └── Dashboard.test.tsx
├── docs/
├── prd.md

```



```

├── architecture.md
├── database-schema.md
├── api-spec.md
├── deployment.md
├── user-manual.md
├── .env.example
├── .gitignore
├── docker-compose.yml          # Local dev: Postgres + Redis + MinIO
├── Dockerfile                 # Production build
├── package.json
├── tsconfig.json
└── README.md

```

4. Environment Configuration

Environment Variables

```

# Application
NODE_ENV=production
PORT=3000
CLIENT_URL=https://fleetspark.syncspherellc.com

# Database
DATABASE_URL=postgresql://fleetspark:PASSWORD@localhost:5432/fleetspark

# Redis
REDIS_URL=redis://localhost:6379

# JWT
JWT_SECRET=your-secret-key-min-32-chars
JWT_REFRESH_SECRET=your-refresh-secret-min-32-chars
JWT_EXPIRY=15m
JWT_REFRESH_EXPIRY=7d

# Email (Resend)
RESEND_API_KEY=re_XXX
EMAIL_FROM=FleetSpark <noreply@fleetspark.syncspherellc.com>

# SMS (Twilio)
TWILIO_ACCOUNT_SID=ACxxx
TWILIO_AUTH_TOKEN=xxx
TWILIO_FROM_NUMBER=+18456134631

# File Storage (MinIO)
S3_ENDPOINT=http://localhost:9000
S3_ACCESS_KEY=minioadmin
S3_SECRET_KEY=minioadmin
S3_BUCKET=fleetspark

# Telemetry
TELEMETRY_API_KEY=fs_telemetry_XXX # API key for GPS devices
TELEMETRY_THROTTLE_MS=30000        # Min interval between telemetry saves

# Default Settings
DEFAULT_DIESEL_PRICE_UGX=5500
DEFAULT_CO2_FACTOR_KG_PER_LITER=2.68

```

5. Docker Compose (Local Development)

```
version: '3.9'

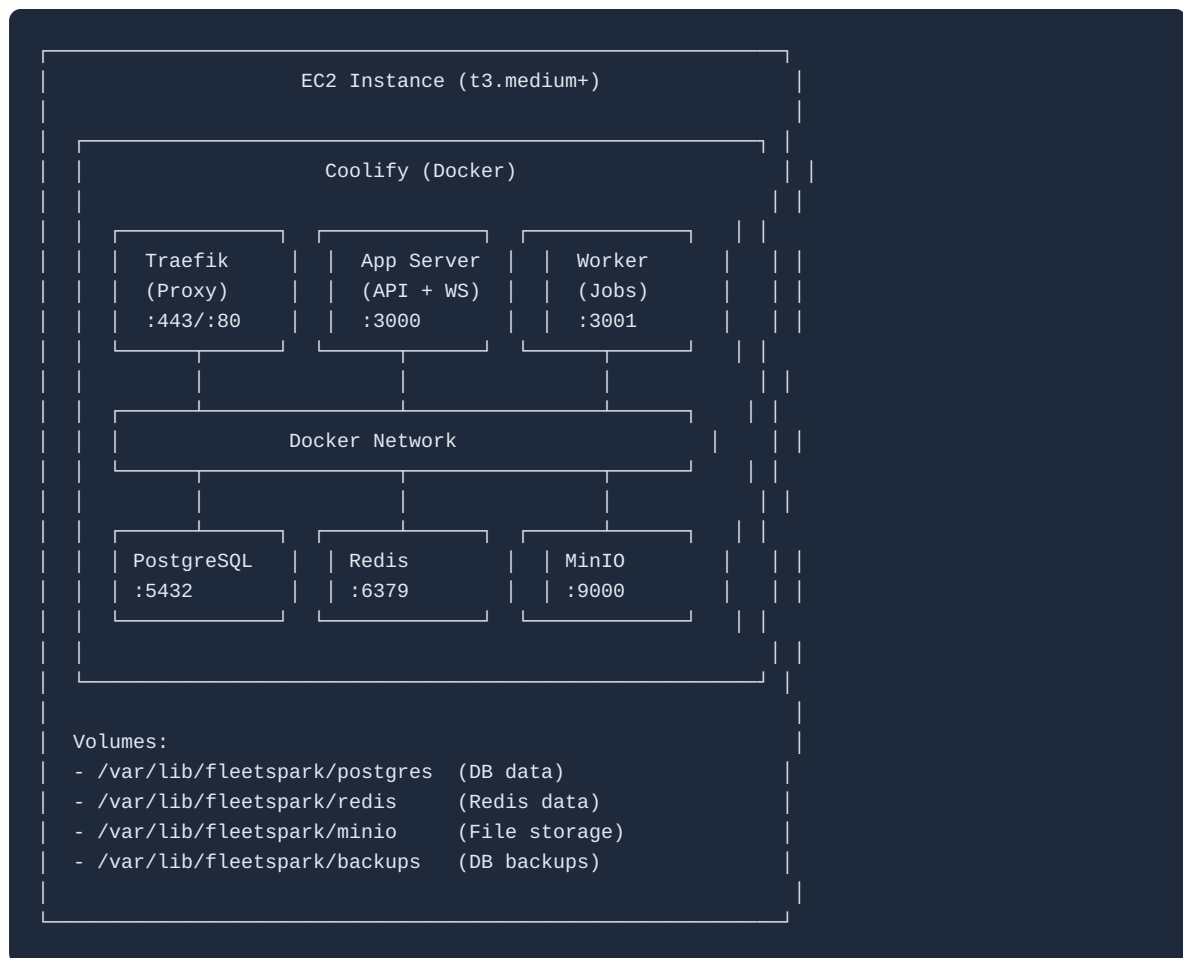
services:
  postgres:
    image: timescale/timescaledb:latest-pg16
    ports:
      - "5432:5432"
    environment:
      POSTGRES_DB: fleetspark
      POSTGRES_USER: fleetspark
      POSTGRES_PASSWORD: fleetspark_dev
    volumes:
      - pgdata:/var/lib/postgresql/data

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"
    volumes:
      - redisdata:/data

  minio:
    image: minio/minio
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: minioadmin
      MINIO_ROOT_PASSWORD: minioadmin
    command: server /data --console-address ":9001"
    volumes:
      - miniodata:/data

volumes:
  pgdata:
  redisdata:
  miniodata:
```

6. Deployment Architecture (Production)



CI/CD Pipeline

```
git push → main
|
▼
GitHub Actions:
1. Run tests (Vitest)
2. Build Docker image
3. Push to GitHub Container Registry
4. Trigger Coolify webhook
|
▼
Coolify:
1. Pull new image
2. Run Prisma migrations
3. Rolling restart (zero-downtime)
4. Health check
5. Notify Slack/Telegram on success/failure
```

7. Security Considerations

Layer	Measure
Transport	HTTPS everywhere (Traefik auto TLS via Let's Encrypt)
Auth	JWT access (15min) + refresh (7d) tokens, bcrypt password hashing
API	Rate limiting (100 req/min per IP, 1000 req/min per API key)
Telemetry	API key authentication, payload validation, throttle per device
Data	Multi-tenant isolation (all queries scoped by org_id)
Headers	Helmet.js (CSP, HSTS, X-Frame-Options, etc.)
Input	Zod validation on all request bodies/params/queries
CORS	Whitelist CLIENT_URL only
Secrets	Environment variables, never in code/repo

8. Scalability Path

MVP (1-50 vehicles, 1-5 orgs)

- Single EC2 instance, all services on one machine
- Handles ~500 telemetry requests/minute comfortably

Growth (50-500 vehicles, 5-25 orgs)

- Separate worker process for telemetry ingestion
- Read replica for PostgreSQL
- CDN for static assets (Cloudflare)

Scale (500+ vehicles, 25+ orgs)

- Separate API and telemetry servers
 - Horizontal scaling with load balancer
 - TimescaleDB continuous aggregation for fast queries
 - Redis Cluster for session/cache
-

9. Monitoring & Alerting

What	How	Threshold
API uptime	Uptime Kuma (self-hosted)	Alert if down >2 min
API response time	pino access logs	Alert if p95 >1s
Error rate	pino error logs	Alert if >5% error rate
DB disk space	Cron check	Alert if >80%
Redis memory	Cron check	Alert if >80%
Failed jobs	BullMQ dashboard	Alert if >3 consecutive failures
SSL cert expiry	Uptime Kuma	Alert if <30 days

10. Cost Estimate (Monthly, Production)

Resource	Spec	Cost
EC2 (t3.medium)	2 vCPU, 4GB RAM	~\$30/mo
EC2 (t3.large) — growth phase	2 vCPU, 8GB RAM	~\$60/mo
PostgreSQL	Managed or self-hosted	\$0 (self-hosted)
Redis	Self-hosted	\$0
MinIO	Self-hosted	\$0
Resend (Email)	Up to 10K emails/mo	\$0 (free tier)
Twilio (SMS)	Pay per SMS	~\$0.05/SMS
Domain + DNS	Cloudflare	\$0 (free tier)
SSL	Let's Encrypt	\$0
Total MVP		~\$30-50/mo
Total Growth		~\$80-120/mo

11. Development Phases

Phase 1: Foundation (Week 1-2)

- Project setup, Docker, CI/CD
- Database schema + migrations
- Auth system (register, login, JWT, roles)
- Basic CRUD: vehicles, drivers
- Telemetry ingestion endpoint

Phase 2: Core Features (Week 3-4)

- Fleet dashboard (map + vehicle list)
- Battery monitoring (gauge, history, alerts)
- Trip tracking
- Alert system (low battery, offline, maintenance)
- WebSocket real-time updates

Phase 3: Polish & Pilot (Week 5-6)

- Reports (PDF/CSV export)
- Settings pages (org, users, alert config)
- Demo data generator + GPS simulator
- Mobile responsive fixes
- Bug fixes, performance optimization
- Deploy to production
- Kiira Motors pilot onboarding